be disruptive if you're trying to read older parts of the chat history. For a more sophisticated user experience, a real-world chat application would enhance the scrolling behavior to

- *Conditional autoscrolling*—Only automatically scrolls to the bottom if the user is already near the bottom of the conversation. If the user scrolls up to review previous messages, autoscrolling should pause to allow uninterrupted reading.
- *Scroll to the Latest button*—When autoscrolling is paused (because the user is viewing older messages and new ones have arrived), a subtle indicator or a Scroll to Latest Message button can appear. Clicking this button manually returns the user to the latest messages.

Listing 2.1 provides a top-level view of the `<ChatPage/>`, `<ChatList/>`, `<AutoScroll/>`, and `<TextArea/>` components. I have omitted some Tailwind CSS classes to improve readability.

NOTE Tailwind CSS is a utility-first framework used in this project for basic styling. While powerful, Tailwind can sometimes result in less readable HTML due to the overuse of CSS classes. For helpful documentation that teaches you how to promote code reusability, see https://tailwindcss.com/docs/reusing-styles. I recommend reviewing it if you plan on working with this framework.

Listing 2.1 The main page component

Form submission logic: useEnterSubmit gives us formRef and onKeyDown to let users submit with Enter.

```
import Textarea from "@/components/ui/textarea";
import ChatList from "@/components/chat/ChatList";

import useEnterSubmit from "@/hooks/use-enter-submit";
import AutoScroll from "@/components/AutoScroll";
import useFocusOnSlashPress from "@/hooks/use-focus-on-slash-press";
import useChatFormSubmit from "@/hooks/use-chat-form-submit";

import { getAssistantResponse } from "@/lib/getAssistantResponse";

const ChatPage = () => {
  // References from custom hooks.
  const { formRef, onKeyDown } = useEnterSubmit();
  const inputRef = useFocusOnSlashPress();

  const { messages, isLoading, handleSubmit, inputValue,
  setInputValue } = useChatFormSubmit(getAssistantResponse);

  const onInputChange = (e) => {
    setInputValue(e.target.value);
  };

  return (
```

Input focus shortcut: useFocusOnSlashPress improves UX by focusing the text box when / is pressed.

Chat state and handlers: useChatFormSubmit centralizes state: messages, loading state, submission handler, and input value.

Input state updates: keeps inputValue synced with the text area